Programação funcional

Aula 06 - Tipos algébricos*

*Baseado no material disponível em https://haskell.pesquisa.ufabc.edu.br/haskell

Criando tipos

- Considere uma função para atualizar o preço de um produto

```
atualiza_preco :: (Integer, String, Double) -> Double -> (Integer, String, Double)
atualiza_preco (id_prod, nome, preco) inflacao = (id_prod, nome, preco*(1 + inflacao))
```

- Agora uma função que calcula o troco:

```
troco :: (Integer, String, Double) -> (Integer, String, Double) -> Double
troco (id_prod, nome, preco) (id_cli, nome_cli, pago) = pago - preco
```

  - É difícil saber a ordem dos parâmetros

Criando tipos

- Em haskell podemos criar apelidos para tipos
  - Com a palavra-chave **type**

```haskell
type Produto = (Integer, String, Double)
type Cliente = (Integer, String, Double)

pago  (_, _, p) = p
preco           = pago

troco :: Produto -> Cliente -> Double
troco produto cliente = pago cliente - preco produto
```

Criando tipos

- Em haskell podemos criar apelidos para tipos
  - Com a palavra-chave **type**

```haskell
type Produto = (Integer, String, Double)
type Cliente = (Integer, String, Double)

pago  (_, _, p) = p
preco           = pago

troco :: Produto -> Cliente -> Double
troco produto cliente = pago cliente - preco produto
```

```haskell
leite = (10, "leite", 2.5)
lucas = (2391, "Lucas", 10)

troco lucas leite
```

Criando tipos

- Em haskell podemos criar apelidos para tipos
  - Com a palavra-chave **type**

```haskell
type Produto = (Integer, String, Double)
type Cliente = (Integer, String, Double)

pago  (_, _, p) = p
preco           = pago

troco :: Produto -> Cliente -> Double
troco produto cliente = pago cliente - preco produto
```

```haskell
leite = (10, "leite", 2.5)
lucas = (2391, "Lucas", 10)

troco lucas leite
```

-7,5

Criando tipos

- String é um caso de **type**

```
type String = [Char]
```

Criando tipos

- Tipos paramétricos

```haskell
type Assoc k v = [(k,v)]

find :: Eq k => k -> Assoc k v -> v
find k t = head [v | (k',v) <- t, k == k']

find 2 [(1,3), (5,4), (2,3), (1,1)]
```

Criando tipos

- Tipos paramétricos

```haskell
type Assoc k v = [(k,v)]

find :: Eq k => k -> Assoc k v -> v
find k t = head [v | (k',v) <- t, k == k']

find 2 [(1,3), (5,4), (2,3), (1,1)]
```

3

Criando tipos “de verdade”

- Não apenas apelidos
  - Tipo Soma
  - Tipo Produto
  - Tipo Recursivo
  - Tipo Exponencial

Criando tipos: Soma

- Como o tipo Bool: `data Bool = True | False`
- Ex.:

```
data Dir = Norte | Sul | Leste | Oeste

type Coord = (Int, Int)
type Passo = Coord -> Coord

data Dir = Norte | Sul | Leste | Oeste

para :: Dir -> Passo
para Norte (x,y) = (x, y + 1)
para Sul   (x,y) = (x, y - 1)
para Leste (x,y) = (x + 1, y)
para Oeste (x,y) = (x - 1, y)
```

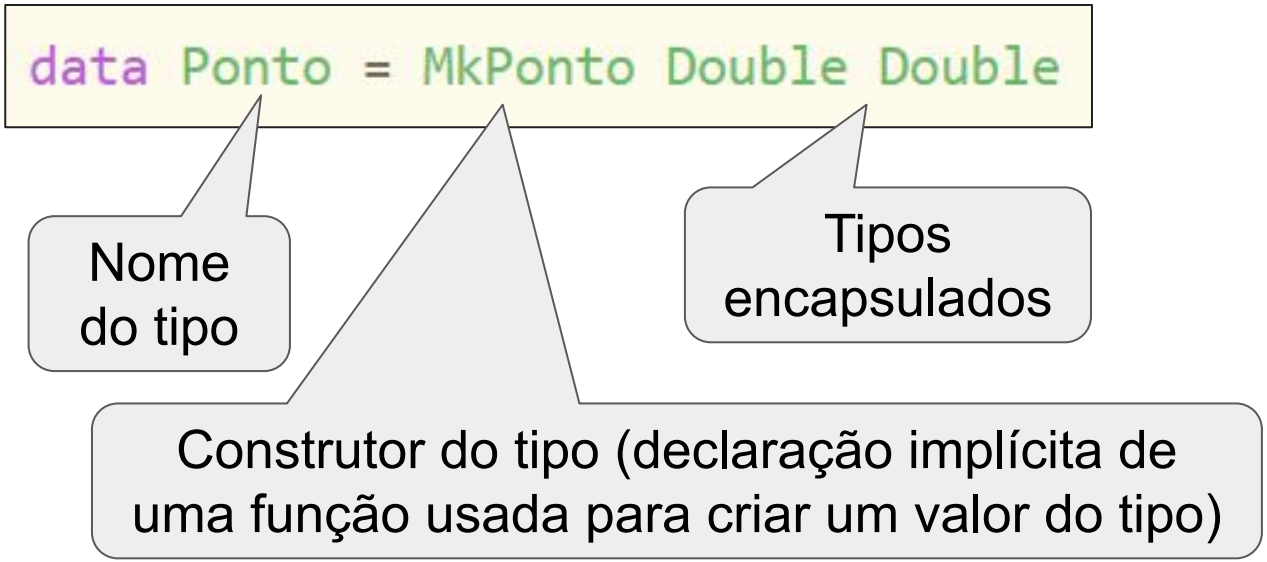
Criando tipos: Soma

```haskell
type Coord = (Int, Int)
type Passo = Coord -> Coord

data Dir = Norte | Sul | Leste | Oeste

caminhar :: [Dir] -> Passo
caminhar []     coord = coord
caminhar (d:ds) coord = caminhar ds (para d coord)
```

Criando tipos: Produto



Criando tipos: Produto

```
data Ponto = MkPonto Double Double
                        deriving (Show)
```

Derivado de outra classe

Tipo imprimível

Criando tipos: Produto

```haskell
data Ponto = MkPonto Double Double
                        deriving (Show)
```

```haskell
dist :: Ponto -> Ponto -> Double
dist (MkPonto x y) (MkPonto x' y') = sqrt $ (x-x')^2 + (y-y')^2

dist (MkPonto 1 2) (MkPonto 1 1)
```

Criando tipos: Produto

- Misturando tipos soma e produto

```haskell
data Forma = Circulo Ponto Double
           | Retangulo Ponto Double Double

-- um quadrado é um retângulo com os dois lados iguais
quadrado :: Ponto -> Double -> Forma
quadrado p n = Retangulo p n n
```

Criando tipos: Produto

- Misturando tipos soma e produto

```haskell
data Forma = Circulo Ponto Double
           | Retangulo Ponto Double Double

-- um quadrado é um retângulo com os dois lados iguais
quadrado :: Ponto -> Double -> Forma
quadrado p n = Retangulo p n n
```

Circulo e Retangulo são funções construtoras

```
> :t Circulo
Circulo :: Ponto -> Double -> Forma

> :t Retangulo
Retangulo :: Ponto -> Double -> Double -> Forma
```

Criando tipos: tipos paramétricos

- Parâmetros de tipo
  - Ex.: **Maybe** do Haskell: `data Maybe a = Nothing | Just a`

```haskell
-- talvez a divisão retorne um Int
maybeDiv :: Int -> Int -> Maybe Int
maybeDiv _ 0 = Nothing
maybeDiv m n = Just (m `div` n)

maybeHead :: [a] -> Maybe a
maybeHead [] = Nothing
maybeHead xs = Just (head xs)
```

Criando tipos: tipos paramétricos

- Parâmetros de tipo
  - Ex.: **Maybe** do Haskell: `data Maybe a = Nothing | Just a`

```haskell
-- talvez a divisão retorne um Int
maybeDiv :: Int -> Int -> Maybe Int
maybeDiv _ 0 = Nothing
maybeDiv m n = Just (m `div` n)

maybeHead :: [a] -> Maybe a
maybeHead [] = Nothing
maybeHead xs = Just (head xs)
```

Criando tipos: tipos paramétricos

- Tipo **Maybe**

```haskell
-- talvez a divisão retorne um Int
maybeDiv :: Int -> Int -> Maybe Int
maybeDiv _ 0 = Nothing
maybeDiv m n = Just (m `div` n)

maybeHead :: [a] -> Maybe a
maybeHead [] = Nothing
maybeHead xs = Just (head xs)
```

```haskell
divComErro :: Int -> Int -> Int
divComErro m n = case (maybeDiv m n) of
                   Nothing -> error "divisão por 0"
                   Just x  -> x
```

Criando tipos: tipos paramétricos

- Tipo **Either** do Haskell:
  - `data Either a b = Left a | Right b`
  - Permite que uma função retorne 2 tipos diferentes

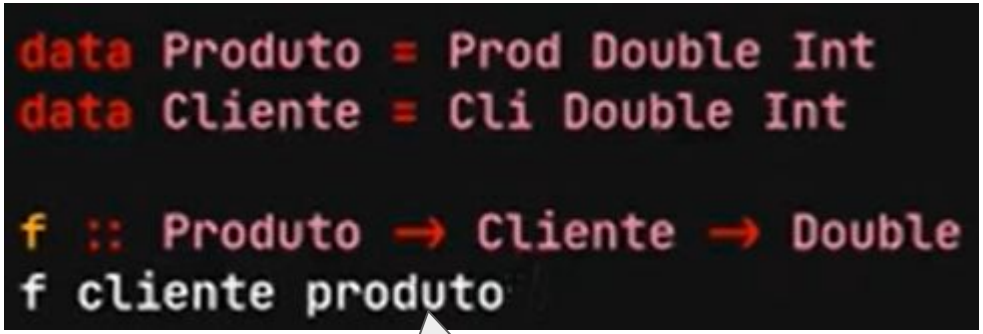
```haskell
-- ou retorna uma String ou um Int
eitherDiv' :: Int -> Int -> Either String Int
eitherDiv' _ 0 = Left "divisao por 0"
eitherDiv' m n = Right (m `div` n)
```

Criando tipos: type *vs* data

- type:
  - Substituição em tempo de compilação
  - Mais eficiente
  - Menos seguro
- data
  - Tipo novo
  - Menos eficiente
  - Mais seguro

Criando tipos: type *vs* data

- type:
  - Substituição em tempo de compilação
  - Mais eficiente
  - Menos seguro
- data
  - Tipo novo
  - Menos eficiente
  - Mais seguro



Criando tipos: newtype

- Meio termo entre **type** e **data**
  - Diferença para o **type**
    - Cria um novo tipo
  - Diferença para o **data**
    - Define um tipo até se compilado
      - Depois é substituído
      - Permite checagem de tipo em tempo de compilação
  - Desvantagem
    - Não pode ser usado com o tipo Soma

Criando tipos: tipo recursivo

- Quando se usa o próprio tipo na definição

```
data Tree a = Leaf a | Node (Tree a) a (Tree a)
```

Criando tipos: tipo recursivo

- Quando se usa o próprio tipo na definição

```
data Tree a = Leaf a | Node (Tree a) a (Tree a)
```

```
t :: Tree Int
t = Node (Node (Leaf 1) 3 (Leaf 4)) 5
         (Node (Leaf 6) 7 (Leaf 9))
```

Criando tipos: tipo recursivo

- Quando se usa o próprio tipo na definição

```
data Tree a = Leaf a | Node (Tree a) a (Tree a)
```

```
t :: Tree Int
t = Node (Node (Leaf 1) 3 (Leaf 4)) 5
         (Node (Leaf 6) 7 (Leaf 9))
```



Criando tipos: tipo recursivo

- Quando se usa o próprio tipo na definição

```
data Tree a = Leaf a | Node (Tree a) a (Tree a)
```

```
t :: Tree Int
t = Node (Node (Leaf 1) 3 (Leaf 4)) 5
         (Node (Leaf 6) 7 (Leaf 9))
```



Criando tipos: tipo recursivo

- Exemplo: árvore

```
data Tree a = Leaf a | Node (Tree a) a (Tree a)
```

- Função **contém**

```
contem :: Eq a => Tree a -> a -> Bool
contem (Leaf y) x      = x == y
contem (Node l y r) x = x == y || l `contem` x
                                || r `contem` x
```

Operador “ou”

Criando tipos: record

- Considere:

```
data Ponto3D = Ponto Double Double Double

coordX (Ponto x _ _) = x
coordY (Ponto _ y _) = y
coordZ (Ponto _ _ z) = z
```

Criando tipos: record

- Considere:

```haskell
data Ponto3D = Ponto Double Double Double

coordX (Ponto x _ _) = x
coordY (Ponto _ y _) = y
coordZ (Ponto _ _ z) = z
```

- Pode ser simplificado pela sintaxe **Record Type**

Criando tipos: record

- Sintaxe **Record Type**
  - Outra forma de definir tipo produto

```
data Ponto3D = Ponto { coordX :: Double
                     , coordY :: Double
                     , coordZ :: Double
                     }
```

Cria as funções **coordX**, **coordY**, **coordZ** que retornam o valor daquele campo

Criando tipos: record

- Sintaxe **Record Type**
  - Outra forma de definir tipo produto

```
data Ponto3D = Ponto { coordX :: Double
                     , coordY :: Double
                     , coordZ :: Double
                     }
```

Cria as funções **coordX**, **coordY**, **coordZ** que retornam o valor daquele campo

```
$> :t coordY
coordY :: Ponto3D → Double
```

Criando tipos: record

- Sintaxe **Record Type**
  - Alterando o tipo árvore:

```haskell
data Tree a = Leaf a | Node (Tree a) a (Tree a)
```

```haskell
data Tree a = Leaf a
            | Node { _right :: Tree a
                   , _value :: a
                   , _left  :: Tree a
                   } deriving Show
```

Referências

- Material dos profs. Emilio Francesquini e Fabrício Olivetti
  - https://haskell.pesquisa.ufabc.edu.br/haskell